BONAFIDE CERTIFICATE

Certified that this project report **"FOOD DELIVERY DATA ANALYSIS"** is the Bonafide work of **"Apoorv Patel, Aditya Sharma, Tanveer Singh"** who carried out the project work under the supervision of the faculty of the Department of Computer Science and Engineering.

Signature of the HoD

Dr. Jaspreet Singh Batth
Head of the Department
Department of CSE
Chandigarh University

Signature of the Supervisor

Mr. Vishal Dutt
Assistant Professor
Department of CSE
Chandigarh University

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 4.1	Level 0 DFD – Context Diagram	25
Figure 4.2	Level 1 DFD – Detailed Process Breakdown	26
Figure 4.3	System Work Flow Diagram	28
Figure 7.1	Chart 1 – Order Status Distribution (Pie Chart)	57
Figure 7.2	Chart 2 – Orders per Restaurant (Bar Chart)	58
Figure 7.3	Chart 3 – Revenue by Cuisine Type (Horizontal Bar)	59
Figure 7.4	Chart 4 – Monthly Order Trend (Line Chart)	60
Figure 7.5	Chart 5 – Orders by Hour of Day (Bar Chart)	61
Figure 7.6	Chart 6 – Delivery Time Distribution (Histogram+KDE)	62
Figure 7.7	Chart 7 – Delivery Time vs Traffic Level (Box Plot)	63
Figure 7.8	Chart 8 – Payment Method Distribution (Donut Chart)	64
Figure 7.9	Chart 9 – Average Rating by Restaurant (Bar Chart)	65
Figure 7.10	Chart 10 – Correlation Heatmap	66
Figure 7.11	Chart 11 – Revenue by City (Bar Chart)	67
Figure 7.12	Chart 12 – Orders by Day & Status (Grouped Bar)	68
Figure 8.1	Model Accuracy: T-Test Result Visualized	72

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1.1	Scope of the Project	12
Table 3.1	Comparison: Existing System vs Proposed System	22
Table 4.1	System Architecture Layers	25
Table 5.1	Dataset Column Descriptions	32
Table 5.2	Sample Dataset Records	35
Table 5.3	Restaurant-Cuisine Mapping	36
Table 5.4	Dataset Statistical Summary	38
Table 6.1	Visualization Overview Table	45
Table 7.1	Development Environment Details	52
Table 7.2	Python Libraries Used	53
Table 7.3	Step-by-Step Code Description	55
Table 8.1	Descriptive Statistics Summary	70
Table 8.2	Statistical Test Results	74
Table 8.3	Key Business Insights Summary	76

ABSTRACT

The food delivery industry in India has witnessed explosive growth over the past decade, driven by smartphone penetration, increasing urban population, and the convenience economy. Platforms such as Swiggy and Zomato collectively process tens of millions of transactions daily, generating vast transactional datasets rich with analytical potential. Despite this abundance of data, many platform operators and restaurant partners lack the tools to extract structured insights that can directly improve operational efficiency, customer satisfaction, and revenue optimization.

This project presents a comprehensive Food Delivery Data Analysis system built using Python and its core data science ecosystem. A realistic synthetic dataset of 1,000 food delivery orders was generated covering 10 restaurants across 10 major Indian cities, 8 cuisine types, 5 payment methods, 4 weather conditions, and 3 traffic levels over a full calendar year (January to December 2023). Each order record contains 19 features capturing financial metrics, logistics performance, customer feedback, and environmental conditions.

A complete, reproducible analytical pipeline was implemented covering six stages: (1) data loading and inspection, (2) data cleaning and preprocessing including missing value treatment and feature engineering, (3) descriptive statistical analysis,
(4) exploratory data analysis through twelve publication-quality visualizations, (5) inferential statistical testing using an independent-samples T-Test and Pearson correlation analysis, and (6) business insight extraction producing eight actionable key metrics.

The twelve visualizations produced span pie charts, bar charts, horizontal bar charts, line charts with area fill, hourly distribution bar charts with peak-hour highlighting, delivery time histograms with KDE overlays, box plots, donut charts, restaurant rating bar charts, correlation heatmaps, city revenue bar charts, and day- of-week grouped bar charts. Each chart is designed to communicate a specific business question clearly and efficiently.

Statistical analysis confirmed that high-traffic conditions produce a statistically significant increase in delivery time compared to low-traffic conditions (T-Test p-value
< 0.0001), while order value shows negligible linear relationship with delivery time
(Pearson r approximately 0.02). Key business metrics extracted include a cancellation rate of approximately 12-13%, peak ordering hours between 19:00 and 21:00, highest revenue cities, and the dominance of UPI as a payment method.

The project demonstrates practical proficiency in Python-based data analysis and visualization and delivers a fully documented, modular pipeline that can be extended with real-time data, machine learning forecasting, and web-based dashboards in future iterations.

Keywords: Food Delivery Analytics, Exploratory Data Analysis (EDA), Python, Pandas, NumPy, Matplotlib, Seaborn, SciPy, Data Visualization, Statistical Analysis, Hypothesis Testing, Business Intelligence, Synthetic Dataset Generation

Chapter 1: INTRODUCTION

1.1 Background

The food delivery industry represents one of the fastest-growing segments of the digital economy globally. In India specifically, the online food delivery market has grown from virtually nothing in 2010 to a multi-billion dollar industry by the mid-2020s. This transformation has been enabled by the widespread adoption of smartphones, the build-out of 4G and 5G networks, and the emergence of dedicated delivery platforms that aggregate restaurant supply with consumer demand through intuitive mobile applications.

Companies such as Swiggy, Zomato, and newer entrants process an enormous volume of daily transactions. Each transaction generates a rich trail of data capturing what was ordered, from which restaurant, by which customer, in which city, at what time, for how much money, paid via what method, in what traffic and weather conditions, how long it took to deliver, and whether the customer was satisfied. This transactional data, accumulated over millions of orders, constitutes an extraordinarily valuable asset for operational optimization.

Traditionally, business analysis in the food delivery sector was performed by dedicated analytics teams using business intelligence (BI) tools such as Tableau or Power BI, connected to proprietary data warehouses. While effective, this approach requires expensive software licenses, specialized training, and creates silos between data generation and insight consumption. The emergence of Python as the lingua franca of data science has democratized access to sophisticated analytical capabilities, enabling engineers, operations managers, and even students to perform rigorous end-to-end analysis on raw data using freely available open-source libraries.

Python's data science ecosystem has matured significantly over the past decade. Pandas provides an intuitive, SQL-like DataFrame abstraction for tabular data manipulation. NumPy underpins all numerical computations with vectorized array operations that are orders of magnitude faster than equivalent Python loops. Matplotlib offers a comprehensive, MATLAB-inspired plotting API that gives users fine-grained control over every visual element. Seaborn extends Matplotlib with higher-level statistical graphics that are particularly useful for understanding distributions, relationships, and categorical patterns. SciPy adds the statistical testing infrastructure necessary to convert descriptive observations into inferential conclusions backed by formal probability theory.

Jupyter Notebook has become the dominant development environment for data analysis projects, combining executable code cells, rich text narrative, mathematical notation, and inline visualizations in a single shareable document. This makes

analyses not just reproducible but also communicable to non-technical stakeholders who can read through the analysis without needing to run code themselves.

This project leverages this entire ecosystem to build a comprehensive food delivery data analysis system. Rather than analyzing data from a live production platform, which would involve significant privacy and access challenges, the project generates a realistic synthetic dataset using a documented generation methodology that preserves the statistical properties and domain logic of real food delivery data.

- ## Problem Statement

Despite the availability of large datasets generated daily by food delivery platforms, a significant gap persists between raw data generation and structured analytical insight. This gap has several dimensions:

- Data Volume and Complexity: Food delivery datasets contain multiple interrelated features spanning financial, temporal, geographic, logistical, and environmental dimensions. Manual inspection of raw data cannot surface the patterns and relationships that exist across these dimensions simultaneously.
- Temporal Pattern Blindness: Without systematic temporal analysis, operators cannot identify whether order volumes peak at lunchtime or dinner time, whether weekends show different cancellation patterns than weekdays, or whether certain months are systematically higher in revenue. These patterns directly drive staffing, inventory, and promotional decisions.
- Logistics Performance Opacity: Delivery time is a critical customer satisfaction driver, but its relationship with controllable and uncontrollable factors (traffic, weather, restaurant preparation time) is not obvious without structured analysis. Without this analysis, service level agreement (SLA) targets may be set arbitrarily.
- Revenue Attribution Blindness: Without proper revenue analysis by restaurant, cuisine, and city, marketing and business development resources cannot be allocated optimally

These challenges collectively highlight the need for a structured, reproducible, and comprehensive analytical pipeline that addresses all major dimensions of food delivery operations in a single integrated project.

• Aim of the Project

The primary aim of this project is to design, implement, and document a comprehensive Python-based data analysis pipeline for food delivery operational data, producing statistically rigorous findings and actionable business insights through a combination of exploratory data analysis, visualization, and formal statistical testing.

The analysis is intended to demonstrate the complete lifecycle of a data analytics project: from raw data generation and cleaning through exploratory visualization to formal statistical inference and business insight extraction. The project serves both as a practical demonstration of Python data science capabilities and as a methodological template for real-world food delivery analytics.

• Objectives of the Project

The specific objectives of this project are:

- To design and implement a synthetic food delivery dataset generator that produces realistic, statistically consistent data covering all major operational dimensions of a food delivery platform.
- To load, inspect, and document the structure and properties of the generated dataset, establishing a clear understanding of the data before analysis begins.
- To perform thorough data cleaning and preprocessing including identification and treatment of missing values, date parsing, and engineered feature creation.
- To compute and present descriptive statistics across all numeric features, providing a statistical foundation for subsequent analysis.
- To produce twelve or more publication-quality visualizations spanning all major analytical dimensions: order distribution, restaurant performance, cuisine revenue, temporal patterns, delivery logistics, customer satisfaction, payment preferences, and geographic revenue.
- To perform formal inferential statistical tests including an independent- samples T-Test comparing delivery times under different traffic conditions and a Pearson correlation analysis examining the relationship between order value and delivery time.

•To extract and present eight key business insight metrics that translate analytical findings into actionable recommendations for platform operators.

•To document the entire project systematically in a format suitable for academic submission, covering background, methodology, implementation, results, and conclusions.

1.5 Scope of the Project

The scope of this project is defined across the analytical dimensions it covers, the tools and technologies it employs, and the potential extensions that could be built upon it.

Scope Area	Description
Dataset	1,000 synthetic food delivery orders with 19 features covering financial, logistics, temporal, geographic, and environmental dimensions
Restaurants	10 restaurants: Spice Garden, Pizza Hub, Burger Barn, Noodle House, Sushi World, Taco Town, Curry Palace, The Grill, Wok Express, Healthy Bites
Cuisines	8 cuisine types: Indian, Italian, American, Chinese, Japanese, Mexican, Continental, Healthy
Cities	10 major Indian cities: Mumbai, Delhi, Bangalore, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, Jaipur, Lucknow
Time Period	Full calendar year 2023 (January 1 to December 31), orders placed between 08:00 and 23:59
Analysis Types	Descriptive statistics, EDA with 12+ visualizations, T-Test, Pearson correlation, business insight extraction
Visualizations	12 chart types: pie, bar, horizontal bar, line, hourly bar, histogram+KDE, box plot, donut, rating bar, heatmap, city bar, grouped bar
Statistical Tests	Independent-samples T-Test (delivery time vs traffic), Pearson correlation (order value vs delivery time)
Programming Language	Python 3.x

Scope Area	Description
Core Libraries	Pandas, NumPy, Matplotlib, Seaborn, SciPy
Development Environment	VS Code with Jupyter extension; also compatible with Jupyter Notebook and Google Colab
Output	12 PNG charts (150 DPI) in /charts/ directory + console-printed statistical report

Future Extension Areas:

•Real-time data ingestion from food delivery platform APIs.

•Interactive web dashboard using Flask or Streamlit.

•Machine learning demand forecasting models.

•Customer segmentation using clustering algorithms.

•Geospatial analysis using mapping libraries.

•Automated daily reporting via scheduled scripts.

6. Limitations

Despite the breadth of analysis covered, the following limitations should be acknowledged:

- Synthetic Data: The dataset is generated programmatically and may not perfectly replicate all statistical properties of a real production platform dataset, particularly in tails of distributions and rare edge cases.
- No Predictive Modelling: The analysis is descriptive and inferential but does not include machine learning models for prediction or forecasting.
- Static Analysis: The pipeline analyzes a fixed snapshot dataset. Real-time streaming analytics are not implemented.
- No Customer Demographics: Individual customer identifiers, demographic information, and repeat-order behaviour are not captured.
- No Geographic Coordinates: Actual delivery distances and GPS-based routing are not modelled; delivery time variations are driven solely by traffic level and weather in the synthetic data.
- No Menu-Level Data: Item-level ordering patterns, popular dishes, and cuisine preference modelling are outside the scope.
- No Competitor Analysis: Cross-platform benchmarking is not possible with a single-platform synthetic dataset.
- No Competitor Analysis: Cross-platform benchmarking is not possible with a single-platform synthetic dataset.

Chapter 2: LITERATURE REVIEW

1. Overview of Food Delivery Data Research

Research into food delivery analytics has evolved in parallel with the industry itself, progressing from early operations research focused on vehicle routing and order batching to contemporary data science work leveraging large-scale transactional datasets to understand consumer behaviour, pricing elasticity, and the effects of external conditions on platform performance.

The academic literature on food delivery analytics broadly falls into three streams: logistics optimization (focused on minimizing delivery time and cost through algorithmic routing), demand forecasting (predicting order volumes by location and time to optimize supply-side readiness), and customer analytics (understanding rating behaviour, repeat purchase patterns, and churn). This project sits at the intersection of all three streams, providing a holistic analytical view rather than focusing on a single operational dimension.

Studies published from 2015 onwards increasingly leverage Python-based tools for food delivery analysis. The ability to generate synthetic datasets with configurable statistical properties has become particularly valuable for academic research, as it enables rigorous methodology demonstrations without requiring access to proprietary commercial data. This approach is well-established in computer science and data science education, where synthetic data generation is used extensively for algorithm benchmarking and pipeline demonstration.

- ### Python for Data Analysis in Industry

Python's rise to dominance in data analytics is well-documented. The 2023 Stack Overflow Developer Survey consistently ranked Python as the most popular programming language, with data science and machine learning as its most common use cases. The language's combination of readability, extensive standard library, and the rich third-party scientific computing ecosystem (NumPy, SciPy, Pandas, Matplotlib, Seaborn, Scikit-learn) makes it uniquely suited to the full data analytics lifecycle.

Wes McKinney, the creator of Pandas, introduced the library in 2008 to address the lack of a high-performance, flexible data manipulation tool in Python. Pandas revolutionized data analysis by providing the DataFrame abstraction, which mirrors the data manipulation capabilities of SQL and spreadsheet software in a programmable, reproducible environment. The groupby-apply paradigm, in particular, enables complex aggregations that would require multiple steps in traditional tools.

The Matplotlib library, created by John Hunter in 2003, provided Python with a comprehensive plotting API inspired by MATLAB. Its fine-grained control over every visual element, combined with its ability to produce publication-quality static graphics, made it the foundation of the Python visualization ecosystem. Seaborn, developed by Michael Waskom and first released in 2012, built a higher-level statistical graphics interface on top of Matplotlib, making it significantly easier to produce complex visualizations such as violin plots, pair plots, and heatmaps with sensible defaults.

SciPy, the scientific computing library underpinning statistical testing in this project, provides a comprehensive suite of statistical functions including parametric and non-parametric hypothesis tests, correlation coefficients, and probability distributions. The scipy.stats module specifically provides the ttest_ind function for independent-samples T-Tests and the pearsonr function for Pearson correlation, both of which are central to the statistical analysis in this project.

- ## Exploratory Data Analysis – Techniques and Best Practices

Exploratory Data Analysis (EDA) was formalized as a discipline by John Tukey in his seminal 1977 book 'Exploratory Data Analysis'. Tukey argued that before fitting formal statistical models to data, analysts should examine the data graphically and descriptively to understand its structure, identify anomalies, and generate hypotheses. This philosophy underpins the analytical approach taken in this project.

Modern EDA practice for tabular datasets typically encompasses four levels of analysis:

- Univariate analysis: examining the distribution of individual variables through histograms, box plots, frequency counts, and summary statistics (mean, median, standard deviation, quartiles, range).
- Bivariate analysis: exploring relationships between pairs of variables through scatter plots, cross-tabulations, grouped bar charts, and correlation coefficients.
- Multivariate analysis: examining relationships across three or more variables simultaneously through heatmaps, pair plots, faceted visualizations, and dimensionality reduction techniques.
- Temporal analysis: examining how variables change over time through line charts, time-series decomposition, and rolling statistics.

For food delivery data specifically, the temporal dimension is particularly important. Order volumes exhibit strong intraday patterns (lunch and dinner peaks), weekly patterns (weekend vs weekday differences), and potentially monthly or seasonal patterns. A rigorous EDA must address all these temporal scales to provide a complete picture of demand dynamics.

The choice of visualization type is a critical design decision in EDA. Different chart types are suited to different analytical questions: pie and donut charts for compositional analysis, bar charts for categorical comparisons, line charts for temporal trends, box plots for distributional comparison across groups, heatmaps for correlation matrices, and histograms with KDE overlays for continuous distributions. This project deliberately employs all these types to ensure that each analytical question is addressed with the most appropriate visual encoding.

2.4 Statistical Hypothesis Testing in Business Analytics

Hypothesis testing provides a formal mathematical framework for distinguishing genuine patterns in data from random variation. In business analytics, hypothesis tests allow analysts to make statements with quantifiable confidence levels about relationships between variables, differences between groups, and the significance of observed trends.

The independent-samples T-Test, also known as Student's t-test, tests whether the means of two independent groups differ significantly. It was developed by William Sealy Gosset (publishing under the pseudonym 'Student') in 1908 for quality control applications in brewing. The test produces a t-statistic (the standardized difference between group means) and a p-value (the probability of observing a difference at least as large as the observed one under the null hypothesis of equal means). A p-value below the conventional threshold of 0.05 is taken as evidence against the null hypothesis.

In the context of this project, the T-Test is applied to test whether mean delivery time under high-traffic conditions is significantly greater than under low-traffic conditions. This is a one-sided test conceptually (we expect higher traffic to increase delivery time, not decrease it), though the two-sided version is implemented via scipy.stats.ttest_ind which reports the two-tailed p-value.

Pearson correlation measures the strength and direction of the linear relationship between two continuous variables. It ranges from -1 (perfect negative linear relationship) to +1 (perfect positive linear relationship), with 0 indicating no linear relationship. The pearsonr function from scipy.stats returns both the correlation coefficient r and the p-value for the test of the null hypothesis that the true correlation is zero.

In the context of food delivery data, the Pearson correlation between order value and delivery time tests whether larger orders (which might involve more items and thus longer preparation times) are associated with longer delivery times. This is a practically important question for understanding the drivers of delivery time variability.

- **Synthetic Data Generation in Research**

The use of synthetic data generation has become increasingly important in data science research and education. When real production data cannot be shared due to privacy regulations, non-disclosure agreements, or the absence of a production system, synthetic data provides a principled alternative that preserves the statistical structure and domain logic of the original domain.

Modern synthetic data generation approaches range from simple rule-based generators (which directly specify the distributions and relationships between variables) to advanced generative models such as variational autoencoders (VAEs) and generative adversarial networks (GANs) that learn to produce data that is statistically indistinguishable from real data.

For this project, a rule-based synthetic data generator was implemented that precisely controls the distributions of all 19 features and embeds domain-specific logic: cancellation probability increases under adverse traffic and weather conditions, delivery time increases with traffic level and rain, and customer ratings are only assigned to delivered (not cancelled) orders. This approach produces data that is internally consistent and analytically tractable while remaining fully transparent in its assumptions.

The use of fixed random seeds (random.seed(42) and np.random.seed(42)) in the generator ensures full reproducibility: every execution of the generator with the same seed produces an identical dataset. This is a critical requirement for academic work where results must be verifiable by reviewers.

- **Prior Work on Food Delivery Analytics**

Several published studies have analyzed food delivery or restaurant data using Python and related tools. Key themes from this literature inform the analytical approach taken in this project:

- Temporal demand analysis: multiple studies have documented the bimodal intraday demand pattern (lunch/dinner peaks) and the weekend uplift effect in food delivery, consistent with the patterns designed into our synthetic dataset and visible in Chart 5 and Chart 12.
- Weather and traffic impact: research consistently finds that adverse weather conditions and high traffic correlate with increased delivery times and order cancellations, consistent with the probabilistic rules embedded in the dataset generator and confirmed by the T-Test results in Chapter 8.
- Payment method evolution: in the Indian context specifically, UPI adoption has grown dramatically since its introduction in 2016, displacing cash and

card payments as the dominant mode for small-value digital transactions including food delivery. This context supports the payment method analysis in Chart 8.

•Customer rating behavior: studies of online review platforms find that customer ratings tend to cluster in the upper range (3.5-5.0 out of 5.0) with relatively rare extreme low ratings, consistent with the 2.0-5.0 uniform distribution used in the synthetic data generator.

- **Research Gaps Addressed by This Project**

Despite significant academic activity in food delivery analytics, this project addresses several specific gaps:

- Most published analyses focus on a single analytical dimension (e.g., only logistics, only customer behavior, or only demand forecasting) without providing a holistic view across all major operational dimensions simultaneously.
- Many studies use proprietary datasets that cannot be shared, making their results difficult to reproduce or validate. This project uses a fully documented open synthetic dataset.
- Published studies rarely provide complete end-to-end code alongside their analysis. This project provides the full Python implementation across two formats (script and Jupyter Notebook).
- The combination of descriptive EDA, formal hypothesis testing, and business insight extraction within a single integrated project is relatively uncommon in the academic literature.

Chapter 3: SYSTEM ANALYSIS

1. Existing System

1. Overview of Existing Food Delivery Analytics Approaches

The existing approach to food delivery data analysis in most organizations relies on a combination of business intelligence (BI) dashboards, spreadsheet-based reporting, and periodic manual analysis by dedicated analytics teams. This approach has evolved incrementally from basic spreadsheet reporting to more sophisticated BI tools, but it still suffers from fundamental limitations that prevent organizations from fully exploiting their transactional data.

In a typical food delivery company's analytics setup, raw transactional data is stored in a relational database (such as PostgreSQL or MySQL) or a cloud data warehouse (such as BigQuery or Amazon Redshift). Periodically, ETL (Extract, Transform, Load) pipelines extract subsets of this data and load it into BI tools such as Tableau, Power BI, or Looker. Business users then interact with pre-built dashboards or create ad-hoc reports using drag-and-drop interfaces.

- **Workflow of the Existing System**
  - Data collection: transactional data is recorded in operational databases by the delivery platform application.
  - ETL pipeline: periodic (hourly, daily) ETL jobs extract, transform, and load data into a reporting database or warehouse.
  - Dashboard creation: BI developers create pre-built dashboards in Tableau, Power BI, or similar tools.
  - Report consumption: business users view dashboards and generate periodic reports (daily, weekly, monthly).
  - Ad-hoc analysis: for questions not covered by pre-built dashboards, analysts use Excel or SQL to perform manual analysis.
  - Insight dissemination: findings are communicated through email reports or presentations.

- **Limitations of the Existing System**
  - High Tool Cost: Enterprise BI tools such as Tableau and Power BI require significant annual licensing fees per user, creating barriers for smaller organizations or academic projects.

•Limited Statistical Depth: Standard BI tools provide descriptive statistics (counts, sums, averages) but do not natively support inferential statistical testing such as T-Tests or correlation analysis.

•Manual Intervention Required: Generating new analyses requires manual configuration of charts and filters. Analyses are not automatically reproducible.

•ETL Dependency: Analysts are dependent on data engineers to build and maintain ETL pipelines and data warehouse schemas. This creates bottlenecks.

•Siloed Insights: Different teams (operations, marketing, finance) typically have separate dashboards, preventing cross-functional analytical views.

•Not Extensible: Adding machine learning or advanced statistical capabilities to a BI tool environment typically requires external tools and significant integration work.

•Version Control Challenges: BI tool configuration is often stored in proprietary formats that are difficult to version control with standard tools like Git.

•Data Privacy Risk: ETL pipelines connecting to production databases create potential security and privacy risks if not carefully managed.

- ## Proposed System

- ### Overview of the Proposed System

The proposed system is a fully Python-based, self-contained analytical pipeline that covers the entire lifecycle from synthetic data generation through to business insight extraction. It is designed to be reproducible, transparent, extensible, and accessible without requiring any proprietary tools or licenses.

The system consists of two Python scripts (generate_dataset.py and food_delivery_analysis.py) and a Jupyter Notebook (Food_Delivery_Analysis.ipynb) that together implement a six-stage analytical pipeline. All dependencies are specified in requirements.txt and can be installed with a single pip command. The entire pipeline runs from a single script execution or through the sequential execution of Jupyter Notebook cells.

- ### How the Proposed System Works

- Stage 1 – Data Generation: The generate_dataset.py script is invoked (manually or automatically by the main analysis script if the CSV does not exist) to create a 1,000-row synthetic food delivery dataset and save it as data/food_delivery_data.csv.

•Stage 2 – Data Loading and Inspection: The main analysis script loads the CSV into a Pandas DataFrame and prints shape, column names, data types, and the first five rows to console.

•Stage 3 – Data Cleaning and Preprocessing: Missing values in Customer_Rating are filled with 0, Order_Date is parsed to datetime, and four derived feature columns are engineered.

•Stage 4 – Descriptive Statistics and EDA: Summary statistics are computed for all numeric columns and printed. Twelve visualizations are generated, saved to /charts/, and closed to release memory.

2.Stage 5 – Statistical Testing: Independent-samples T-Test and Pearson correlation are computed and results printed with interpretation.

3.Stage 6 – Business Insights: Eight key business metrics are computed and printed as a formatted summary.

2. Advantages of the Proposed System

2. Zero Cost: All tools and libraries used are open-source with no licensing fees. The analysis can be run on any machine with Python installed.
3. Fully Reproducible: Fixed random seeds in the data generator and deterministic analytical code ensure identical results on every run.
4. End-to-End Coverage: The system covers the complete analytical lifecycle from raw data generation to business-ready insight extraction in a single integrated pipeline.
5. Statistical Rigor: The system goes beyond descriptive statistics to include formal hypothesis testing, providing analytically rigorous rather than just descriptive conclusions.
6. Portable: The system runs on Windows, macOS, and Linux with no platform- specific dependencies. It is also compatible with Google Colab for cloud- based execution.
7. Version Controllable: All code is in plain text Python files that can be versioned with Git, enabling collaborative development and change tracking.
8. Extensible: The modular design allows individual analysis components to be enhanced, replaced, or extended without affecting other components.
9. Interactive Format Available: The Jupyter Notebook version enables cell-by- cell execution and inline visualization for interactive exploration.
10. Self-Documenting: The Jupyter Notebook format combines code, output, and narrative explanation in a single document that serves as both implementation and documentation.

- ## Limitations of the Proposed System
  - Static Analysis: The system analyzes a fixed snapshot of data rather than a live data stream. Real-time monitoring is not supported.
  4. No Web Interface: Results are output to console and PNG files. There is no web-based dashboard or interactive visualization.
  5. No Automated Scheduling: The pipeline must be triggered manually. Automated daily or weekly reporting requires external scheduling tools.
  6. Synthetic Data Limitations: The dataset is generated according to specified rules and may not capture all real-world complexity.
  - No Database Integration: The system reads from CSV files rather than connecting directly to a database or API.

3.3 Comparison: Existing vs Proposed System

Parameter Tools)	Existing System (BI Pipeline)	Proposed System (Python
Cost	High (enterprise licenses)	Zero (open-source)
Setup Complexity	High (ETL pipelines, server setup)	Low (pip install + run script)
Analysis Speed	Hours to days (manual)	Seconds (automated pipeline)
Reproducibility	Low (manual steps)	High (fixed seeds, script-based)
Statistical Depth	Descriptive only	Descriptive + Inferential
Visualization Flexibility	Limited by tool	Unlimited (full Matplotlib/Seaborn)
Version Control	Difficult	Native Git support
Portability	Platform-specific	Cross-platform (Win/Mac/Linux)
Extensibility	Limited (vendor lock-in)	Unlimited (open Python)
Scalability	BI tool dependent	Scales with Pandas/Dask/Spark
Documentation	Separate from analysis	Embedded in Jupyter Notebook

Parameter	Existing System (BI Tools)	Proposed System (Python Pipeline)
Machine Learning Ready	No (requires integration)	Yes (scikit-learn directly importable)

4. Feasibility Analysis

1. Technical Feasibility

The system uses Python 3.x and six open-source libraries (pandas, numpy, matplotlib, seaborn, scipy, jupyter). All of these are actively maintained, widely used, and available for free on all major operating systems. The analysis runs on standard consumer hardware with 4GB RAM or more in under 60 seconds. No GPU, specialized hardware, or cloud infrastructure is required. The system has been tested on both Windows (the original development environment, as seen from the VS Code screenshots) and is compatible with macOS (the user's platform) and Linux.

- **Economic Feasibility**

The economic feasibility of the system is excellent. All software components are free and open-source with no licensing costs. The only resource required is a standard personal computer or laptop. There are no ongoing operational costs (no cloud hosting, no database licenses, no BI tool subscriptions). The dataset is generated synthetically, eliminating any data acquisition costs.

- **Operational Feasibility**

The operational feasibility of the system is high. The analysis pipeline requires only two user actions: running pip install -r requirements.txt once to install dependencies, and then either running python food_delivery_analysis.py or opening the Jupyter Notebook. The Jupyter Notebook version is particularly accessible for non- developers, as it allows cell-by-cell execution with inline output and narrative explanation. The auto-generation of the dataset (if not present) eliminates the need for separate data preparation steps.

- **Schedule Feasibility**

The project was developed within the academic semester schedule. The modular structure of the codebase, with clearly separated sections for data generation, loading, cleaning, visualization, and statistical analysis, allowed parallel development and incremental testing. The use of a synthetic dataset eliminated the dependency on external data procurement timelines.

5. Functional and Non-Functional Requirements

1. Functional Requirements

- The system shall generate a 1,000-row synthetic food delivery dataset with 19 columns covering all major operational dimensions.
- The system shall load the dataset from CSV format and print shape, column names, data types, and sample records.
- The system shall identify and report missing values and duplicate rows.
- The system shall clean the dataset by filling missing ratings, parsing date columns, and engineering derived features.
- The system shall compute and print descriptive statistics for all numeric columns.
- The system shall generate twelve visualization charts and save each as a PNG file in the /charts/ directory.
- The system shall perform an independent-samples T-Test comparing delivery time between low and high traffic conditions and print the result with interpretation.
- The system shall compute the Pearson correlation between order value and delivery time and print the result.
- The system shall compute and print eight key business insight metrics.

• Non-Functional Requirements

- Performance: The complete analysis pipeline shall run in under 60 seconds on standard consumer hardware.
- Reproducibility: Running the pipeline multiple times with the same data shall produce identical outputs.
- Portability: The system shall run on Windows, macOS, and Linux without platform-specific modifications.
- Maintainability: Code shall be organized into clearly labeled sections with inline comments.
- Readability: Chart titles, axis labels, and legends shall be clear and self- explanatory.
- Chart Quality: All charts shall be saved at 150 DPI minimum for print quality.

• Input and Output Description

• Input

The primary input to the analysis system is the CSV file data/food_delivery_data.csv, generated by generate_dataset.py. This file contains 1,000 rows and 19 columns. If the file does not exist when the main analysis script is run, the generator is invoked automatically to create it. The generator has no external data input requirements and creates the dataset entirely from its internal configuration.

The generate_dataset.py script accepts configuration through module-level constants at the top of the file: NUM_RECORDS (number of orders to generate), RESTAURANTS (list of restaurant names), CUISINES (mapping of restaurant names to cuisine types), CITIES (list of city names), PAYMENT_METHODS (list of payment methods), ORDER_STATUS (list of status values), WEATHER (list of weather conditions), and TRAFFIC (list of traffic levels). These can be modified to change the characteristics of the generated dataset.

- **Output**

The system produces two categories of output:

- Visual Output: Twelve PNG chart files saved to the /charts/ directory, each at 150 DPI resolution with tight bounding boxes. File names follow the pattern NN_descriptive_name.png (e.g., 01_order_status_pie.png, 10_correlation_heatmap.png).
- Console Output: Structured text output covering dataset shape and column information, data type summary, missing value counts before and after cleaning, descriptive statistics for numeric columns, categorical distribution summaries, T-Test results with test statistic and p-value, Pearson correlation results, and the key business insights summary block.

- **Constraints**

- Memory Constraint: For datasets significantly larger than 100,000 rows, Pandas in-memory processing may become a bottleneck. Dask or PySpark would be required for very large datasets.
- Visualization Constraint: Matplotlib produces static PNG outputs. Interactive visualizations for end-user consumption would require Plotly, Bokeh, or Altair.
- Synchronous Execution: The pipeline runs sequentially. Each stage must complete before the next begins. Parallel execution of independent chart generation would reduce total runtime.
- Platform Dependency: While Python is cross-platform, some display behavior (font rendering, DPI scaling) may differ slightly between operating systems.

- **Assumptions**

•The food delivery dataset follows the statistical properties encoded in the generator: cancellation probability of 10-18% depending on conditions, delivery time baseline of 20-40 minutes with additive delays for traffic and weather, and customer ratings uniformly distributed between 2.0 and 5.0.

•The analysis environment has Python 3.x installed with pip access to the Python Package Index (PyPI) for library installation.

•The /charts/ and /data/ directories are writable on the local filesystem.

•The system is run interactively (not in a headless server environment without display) when using the Jupyter Notebook version.

Chapter 4: SYSTEM DESIGN

4.1 System Architecture

The Food Delivery Data Analysis system is designed as a sequential six-layer pipeline architecture. Each layer receives its input from the preceding layer and passes its output to the following layer. The layers are implemented as clearly demarcated sections within the main analysis script, enabling individual sections to be run independently for debugging or incremental development.

Layer	Name	Description	Key Tools
Layer 0	Import and Configuration	Imports all required libraries, sets plot style and color palette, suppresses warnings, and creates output directories.	import, os.makedirs, plt.style.use, sns.set_palette
Layer 1	Data Generation	generate_dataset.py creates a 1,000-row synthetic CSV with 19 columns using configurable parameters and fixed random seeds. Invoked automatically if CSV is absent.	random, numpy.random, pandas, datetime
Layer 2	Data Ingestion	Loads CSV into Pandas DataFrame using pd.read_csv(). Prints shape, column names, data types, and first 5 rows.	pd.read_csv, df.shape, df.dtypes, df.head
Layer 3	Preprocessing	Fills missing values, parses date columns, engineers 4 derived features (Month_Num, Is_Weekend, Is_Delivered, Discount_Pct).	fillna, to_datetime, dt.month, isin, astype
Layer 4	EDA and Visualization	Computes descriptive statistics. Generates 12 charts using matplotlib and seaborn. Saves each chart as PNG to /charts/.	df.describe, plt.subplots, sns.heatmap, fig.savefig
Layer 5	Statistical Analysis	Performs T-Test (delivery time: low vs high traffic) and Pearson correlation (order value vs delivery time).	scipy.stats.ttest_ind, pearsonr

Layer	Name	Description	Key Tools
Layer 6	Business Insights	Extracts 8 key business metrics using groupby, idxmax, mean, and sum operations. Prints formatted summary.	groupby, value_counts, mean, sum, idxmax

2. Data Flow Diagram

1. Level 0 DFD – Context Diagram

The Level 0 DFD provides the highest-level view of the system. It shows the single external entity (the analyst/user), the single process (the Food Delivery Analysis System), and the data flows between them.

Entity / Process	Description
External Entity: Analyst/User	Triggers the pipeline by running the analysis script or Jupyter Notebook cells
Input Data Flow	food_delivery_data.csv (generated automatically if absent): 1,000 rows, 19 columns
Process: Food Delivery Analysis System	Complete six-stage analytical pipeline: generate, load, clean, analyze, visualize, insight-extract
Output Data Flow 1	12 PNG chart files in /charts/ directory (visual analytical outputs)
Output Data Flow 2	Console report: statistical summaries, test results, business insights

4.2.2 Level 1 DFD – Detailed Process Breakdown

The Level 1 DFD decomposes the main process into its constituent stages, showing internal data flows between stages.

Process ID	Process Name	Input	Output	Storage
P1	Generate Dataset	Configuration constants (restaurants, cities, etc.)	food_delivery_data.csv	data/ directory
P2	Load and Inspect	food_delivery_data.csv	DataFrame in memory, console inspection output	RAM
P3	Clean and Preprocess	Raw DataFrame	Cleaned DataFrame with 4 derived columns	RAM
P4	Descriptive Statistics	Cleaned DataFrame	Console stats output	Console
P5	Generate Visualizations	Cleaned DataFrame	12 PNG chart files	charts/ directory
P6	Statistical Tests	Delivered-only filtered DataFrame	T-Test and correlation results on console	Console
P7	Extract Insights	Cleaned + Delivered DataFrames	8-metric business insights block on console	Console

4.3 Work-Flow Diagram

The workflow diagram describes the complete execution sequence of the system from initial trigger to final output. Each step represents a discrete operation in the pipeline.

Step	Action	Condition / Notes
Step 1	Analyst runs python food_delivery_analysis.py or opens Jupyter Notebook	Entry point of the system
Step 2	System checks for data/food_delivery_data.csv	File existence check via os.path.exists()

Step	Action	Condition / Notes
Step 3	If CSV absent: import and run generate_dataset.py	Automatic data generation; skipped if CSV exists
Step 4	Load CSV into df DataFrame	pd.read_csv(); prints shape and first 5 rows
Step 5	Print column names, data types, and null counts	Initial data quality assessment
Step 6	Fill Customer_Rating NaN values with 0	Cancelled/Pending orders have no rating
Step 7	Parse Order_Date to datetime	Enables temporal feature extraction
Step 8	Engineer derived columns: Month_Num, Is_Weekend, Is_Delivered, Discount_Pct	Enriches dataset for downstream analysis
Step 9	Print post-cleaning null counts	Confirms no remaining missing values
Step 10	Compute df[numeric_cols].describe()	Generates descriptive statistics summary
Step 11	Print value_counts for Order_Status, Restaurant_Name, City	Categorical distribution summaries
Step 12	Generate Chart 1: Order Status Pie Chart	Save to charts/01_order_status_pie.png
Step 13	Generate Chart 2: Orders per Restaurant Bar Chart	Save to charts/02_orders_per_restaurant.png
Step 14	Generate Chart 3: Revenue by Cuisine Horizontal Bar	Save to charts/03_revenue_by_cuisine.png
Step 15	Generate Chart 4: Monthly Order Trend Line Chart	Save to charts/04_monthly_order_trend.png

Step	Action	Condition / Notes
Step 16	Generate Chart 5: Orders by Hour Bar Chart	Save to charts/05_orders_by_hour.png
Step 17	Generate Chart 6: Delivery Time Distribution Histogram+KDE	Save to charts/06_delivery_time_distribution.png
Step 18	Generate Chart 7: Delivery Time vs Traffic Box Plot	Save to charts/07_delivery_time_vs_traffic.png
Step 19	Generate Chart 8: Payment Method Donut Chart	Save to charts/08_payment_method_donut.png
Step 20	Generate Chart 9: Average Rating by Restaurant Bar Chart	Save to charts/09_avg_rating_by_restaurant.png
Step 21	Generate Chart 10: Correlation Heatmap	Save to charts/10_correlation_heatmap.png
Step 22	Generate Chart 11: Revenue by City Bar Chart	Save to charts/11_revenue_by_city.png
Step 23	Generate Chart 12: Orders by Day and Status Grouped Bar	Save to charts/12_orders_by_day_and_status.png
Step 24	T-Test: Compare Delivery Time (Low vs High Traffic)	scipy.stats.ttest_ind(); print result and interpretation
Step 25	Pearson Correlation: Order Value vs Delivery Time	scipy.stats.pearsonr(); print r and p-value
Step 26	Compute and print key business metrics	8 metrics: cancellation rate, peak hour, top city, etc.
Step 27	Print completion message	ANALYSIS COMPLETE! All charts saved to /charts/

4. System Components

1. generate_dataset.py

This standalone Python script is responsible for creating the synthetic food delivery dataset. It defines the domain entities (restaurants, cities, cuisines, payment methods, weather conditions, traffic levels) as module-level constants and implements a generate_food_delivery_data(n) function that produces n order records as a Pandas DataFrame.

Key generation logic includes: order date and time generation using random offsets from a base date, traffic and weather-dependent delivery time calculation, order value generation with a small probability of high-value outliers, discount calculation as a fraction of order value, cancellation probability determination based on traffic and weather conditions, and customer rating assignment only for delivered orders.

- **food_delivery_analysis.py**

This is the main analysis script that implements all six stages of the analytical pipeline. It is organized into clearly numbered and labeled sections (Section 0 through Section 6) that correspond exactly to the pipeline stages. Each section is self- contained and prints progress indicators to the console. The save() helper function provides a consistent interface for writing charts to the /charts/ directory.

- **Food_Delivery_Analysis.ipynb**

This Jupyter Notebook implements the same analytical pipeline as food_delivery_analysis.py but in an interactive, cell-by-cell format. Each code cell corresponds to a logical unit of analysis and is preceded by a Markdown cell providing narrative explanation. Charts are rendered inline rather than saved to files. The notebook uses %matplotlib inline magic to enable inline visualization within the notebook interface.

The notebook version is particularly suited to academic demonstration and presentation, as it combines executable code with narrative explanation and inline visualizations in a format that can be exported to HTML or PDF for submission. The screenshots provided with this project show the notebook running in VS Code's Jupyter extension.

- **requirements.txt**

This file specifies minimum version constraints for all six dependencies: pandas>=1.5.0, numpy>=1.23.0, matplotlib>=3.6.0, seaborn>=0.12.0, scipy>=1.9.0, jupyter>=1.0.0, notebook>=6.5.0, and ipykernel>=6.0.0. These version constraints ensure compatibility while allowing newer patch versions to be installed.

- **Pseudo Code**

The following pseudo code describes the high-level logic of the complete analysis pipeline:

```
•BEGIN
•IF data/food_delivery_data.csv NOT EXISTS:
•CALL generate_food_delivery_data(1000)
•SAVE result to data/food_delivery_data.csv
•END IF
•LOAD data/food_delivery_data.csv INTO DataFrame df
•PRINT df.shape, df.dtypes, df.head(5)
•FILL df['Customer_Rating'] NaN WITH 0
1.PARSE df['Order_Date'] AS datetime
2.CREATE df['Month_Num'] = df['Order_Date'].month
3.CREATE df['Is_Weekend'] = 1 if Saturday or Sunday else 0
4.CREATE df['Is_Delivered'] = 1 if Order_Status == 'Delivered' else 0
5.CREATE df['Discount_Pct'] = (Discount / Order_Value) * 100
6.PRINT df[numeric_cols].describe()
7.FOR each of 12 chart specifications:
8.CREATE figure and axes
9.PLOT chart using matplotlib/seaborn
10.SAVE figure to charts/NN_name.png at 150 DPI
11.CLOSE figure
12.END FOR
13.SET delivered = df WHERE Order_Status == 'Delivered'
14.PERFORM T-Test: ttest_ind(low_traffic_times, high_traffic_times)
15.PRINT t-statistic, p-value, and interpretation
16.PERFORM Pearson: pearsonr(order_values, delivery_times)
17.PRINT correlation coefficient r and p-value
18.COMPUTE AND PRINT 8 business insight metrics
19.PRINT 'ANALYSIS COMPLETE'
20.END
```

Chapter 5: DATASET DESCRIPTION

1. Dataset Source

The dataset used in this project is a synthetic food delivery dataset generated programmatically using the generate_dataset.py Python script. The decision to use a synthetic dataset rather than a real-world dataset was made for several reasons: synthetic data is fully reproducible (any analyst can regenerate an identical dataset), it does not involve privacy or data governance concerns, it can be configured to exhibit the statistical properties most suitable for demonstrating a comprehensive range of analytical techniques, and it eliminates the practical barriers of data procurement and cleaning that would otherwise be required.

The generation methodology is fully documented and transparent. The script defines all domain entities explicitly as module-level constants, implements realistic domain logic (traffic and weather affect delivery times and cancellation rates), and uses fixed random seeds to ensure reproducibility. The generated dataset is saved as a standard CSV file in the data/ subdirectory.

- Source: Synthetically generated using Python (generate_dataset.py)
- Format: CSV (Comma-Separated Values)
- File Path: data/food_delivery_data.csv
- Total Records: 1,000 orders
- Total Columns: 19 features
- Time Period: January 1, 2023 to December 31, 2023
- Random Seed: 42 (both random and numpy.random)

• Dataset Overview

The dataset models a food delivery platform operating across 10 major Indian cities over a full calendar year. Each record represents one complete order transaction from placement through delivery (or cancellation). The 19 features capture every major dimension of the transaction: identity (Order_ID), temporal (Order_Date, Order_Time, Hour, Day_of_Week, Month), geographic (City), restaurant and product (Restaurant_Name, Cuisine_Type), financial (Order_Value, Discount, Delivery_Fee, Final_Amount), payment (Payment_Method), logistics (Delivery_Time_min, Order_Status), customer feedback (Customer_Rating), and environmental (Weather, Traffic_Level).

Column Name	Data Type	Range / Categories	Description
Order_ID	String	ORD00001 to ORD01000	Unique identifier for each order
Order_Date	String (parsed to datetime)	2023-01-01 to 2023-12-31	Date on which the order was placed
Order_Time	String (HH:MM format)	08:00 to 23:59	Time at which the order was placed
Restaurant_Name	String (categorical)	10 values	Name of the restaurant fulfilling the order
Cuisine_Type	String (categorical)	8 values	Type of cuisine (derived from Restaurant_Name)
City	String (categorical)	10 values	Indian city where the order was placed
Order_Value	Float	~Rs.100 to Rs.2000	Original order amount before discount
Discount	Float	Rs.0 to ~Rs.600	Discount applied (0-30% of Order_Value)
Delivery_Fee	Integer	0, 20, 30, 40, 50	Delivery charge added to the order
Final_Amount	Float	Derived	Order_Value minus Discount (amount actually paid)
Payment_Method	String (categorical)	5 values	Mode of payment used by the customer
Delivery_Time_min	Integer	~20 to ~75 minutes	Actual delivery duration in minutes
Order_Status	String (categorical)	Delivered, Cancelled, Pending	Final status of the order

Column Name	Data Type	Range / Categories	Description
Customer_Rating	Float (nullable)	2.0 to 5.0 or NaN	Customer satisfaction rating (only for Delivered orders)
Weather	String (categorical)	Sunny, Rainy, Cloudy, Windy	Weather condition at time of order
Traffic_Level	String (categorical)	Low, Medium, High	Traffic level at time of order
Day_of_Week	String (categorical)	Monday to Sunday	Day of the week derived from Order_Date
Month	String (categorical)	January to December	Month name derived from Order_Date
Hour	Integer	8 to 23	Hour of the day (24-hour format)

5.3 Sample Dataset Records

The following table shows a representative sample of 8 records from the generated dataset to illustrate the structure and typical values of the data:

Order_ID	Date	Restaurant	City	Order_Value	Discount	Status	Rating
ORD00001	2023-03-15	Spice Garden	Mumbai	Rs.485.50	Rs.97.10	Delivered	4.2
ORD00042	2023-06-22	Pizza Hub	Delhi	Rs.312.80	Rs.0.00	Delivered	3.8
ORD00107	2023-01-08	Burger Barn	Bangalore	Rs.678.20	Rs.135.64	Cancelled	N/A
ORD00234	2023-09-11	Sushi World	Chennai	Rs.1240.00	Rs.248.00	Delivered	4.7
ORD00356	2023-11-30	Curry Palace	Kolkata	Rs.195.60	Rs.39.12	Pending	N/A

Order_ID	Date	Restaurant	City	Order_Value	Discount	Status	Rating
ORD00489	2023-07-04	Noodle House	Pune	Rs.423.90	Rs.84.78	Delivered	3.5
ORD00612	2023-04-17	Healthy Bites	Hyderabad	Rs.288.40	Rs.0.00	Delivered	4.1
ORD00799	2023-12-25	Taco Town	Ahmedabad	Rs.356.70	Rs.71.34	Delivered	3.9

5.4 Restaurant and Cuisine Reference

The dataset covers 10 restaurants across 8 cuisine types. Each restaurant serves exactly one cuisine type as defined in the CUISINES mapping constant in generate_dataset.py:

Restaurant Name	Cuisine Type	Notes
Spice Garden	Indian	Represents mainstream North Indian cuisine
Pizza Hub	Italian	Focuses on pizzas and pasta
Burger Barn	American	Fast food: burgers, fries, shakes
Noodle House	Chinese	Noodles, fried rice, Manchurian
Sushi World	Japanese	Sushi, ramen, bento boxes
Taco Town	Mexican	Tacos, burritos, quesadillas
Curry Palace	Indian	South Indian and Mughlai cuisine
The Grill	Continental	Grilled meats, salads, continental fare
Wok Express	Chinese	Quick-service Chinese, wok dishes
Healthy Bites	Healthy	Salads, wraps, low-calorie meals

5.5 City Reference

The 10 Indian cities covered in the dataset represent a geographically and economically diverse cross-section of the

City	Region	Notes
Mumbai	West India	Financial capital; high-volume market
Delhi	North India	National capital; high order volumes
Bangalore	South India	Tech hub; high UPI adoption
Hyderabad	South India	Growing tech city; Biryani market
Chennai	South India	South Indian cuisine focus
Kolkata	East India	Strong street food culture
Pune	West India	Young professional demographic
Ahmedabad	West India	Growing delivery market
Jaipur	North India	Heritage city with diverse cuisine
Lucknow	North India	Famous for Awadhi cuisine

6. Dataset Generation Logic

The generate_dataset.py script implements the following domain logic to produce a realistic dataset:

- ### Temporal Generation

Order dates are generated by adding a random number of days (0 to 364) and a random number of hours (8 to 23) and minutes (0 to 59) to a base date of January 1, 2023. This produces a uniform distribution of orders across the calendar year with operating hours from 08:00 to 23:59.

- ### Financial Generation

Order values are drawn from a uniform distribution between Rs.100 and Rs.800. With 5% probability, an order is designated as a high-value outlier and its value is redrawn from a uniform distribution between Rs.1,000 and Rs.2,000.

Discounts are calculated as a uniform random fraction between 0% and 30% of the order value. Delivery fees are selected uniformly from the set {0, 20, 30, 40, 50} rupees. Final amount equals order value minus discount.

- **Delivery Time Generation**

Base delivery time is drawn from a uniform distribution between 20 and 40 minutes. If the traffic level is High, a random additional delay between 10 and 20 minutes is added. If the weather is Rainy, a further random delay between 5 and 15 minutes is added. This produces the observed positive correlation between traffic level and delivery time that is confirmed by the T-Test in Chapter 8.

- **Order Status Generation**

A cancellation probability is set at 10% by default, increasing to 18% if the traffic level is High or the weather is Rainy. A uniform random number is drawn: if it falls below the cancellation probability, the order is Cancelled; if it falls between the cancellation probability and the cancellation probability plus 3%, the order is Pending; otherwise, the order is Delivered.

- **Customer Rating Generation**

Customer ratings are only assigned to Delivered orders. Cancelled and Pending orders have NaN as their rating. Ratings for delivered orders are drawn from a uniform distribution between 2.0 and 5.0 and rounded to one decimal place.

- **Dataset Statistical Summary**

Property	Value
Total Records	1,000 orders
Date Range	January 1, 2023 to December 31, 2023
Order Value: Mean (approx.)	Rs.470-490
Order Value: Range	Rs.100 to Rs.2,000 (with ~5% high-value outliers)
Discount: Mean (approx.)	Rs.95-105
Final Amount: Mean (approx.)	Rs.370-390

Property	Value
Delivery Time: Mean (approx.)	42-48 minutes
Delivery Time: Range	~20 to ~75 minutes
Customer Rating: Mean (approx.)	3.4-3.6 out of 5.0
Customer Rating: Range	2.0 to 5.0
Approximate Delivery Rate	~87-88%
Approximate Cancellation Rate	~11-13%
Approximate Pending Rate	~1-2%
Missing Values (pre-cleaning)	Customer_Rating: ~120-130 NaN values (non-delivered orders)
Missing Values (post-cleaning)	0 across all columns
Duplicate Rows	0

5.8 Derived Features

Four derived features are engineered during the preprocessing stage to enrich the analytical capabilities of the dataset:

Feature Name	Formula	Data Type	Purpose
Month_Num	Order_Date.dt.month	Integer (1-12)	Enables numeric ordering and arithmetic on months; used for chronological sorting in the monthly trend chart
Is_Weekend	1 if Day_of_Week in ['Saturday','Sunday'] else 0	Binary Integer	Enables weekend vs weekday segmentation in correlation analysis and business insights

Feature Name	Formula	Data Type	Purpose
Is_Delivered	1 if Order_Status == 'Delivered' else 0	Binary Integer	Enables correlation analysis between delivery success and financial metrics
Discount_Pct	round((Discount / Order_Value) * 100, 2)	Float	Normalizes discount as a percentage of order value for cross-order comparison; reported in the business insights section

Chapter 6: METHODOLOGY

1. Overview of the Analytical Approach

The analytical methodology follows a six-stage structured pipeline that progresses logically from raw data through increasingly sophisticated levels of analysis. This approach mirrors the standard data science lifecycle: understand, clean, explore, model, interpret, and communicate. Each stage builds on the outputs of the previous stage, ensuring that the analysis is coherent and that findings at each level are grounded in the data quality established in earlier stages.

The methodology is intentionally comprehensive, touching all major analytical dimensions of food delivery data: compositional analysis (what proportion of orders are delivered vs cancelled), comparative analysis (which restaurant, cuisine, or city generates the most revenue), temporal analysis (when are orders placed, and how does volume change over time), distributional analysis (what is the distribution of delivery times), relational analysis (how do variables correlate with each other), and inferential analysis (are observed differences statistically significant).

• Stage 1: Data Loading and Inspection

The first methodological step is to load the dataset and establish a clear understanding of its structure before any analysis begins. This inspection-first approach prevents common analytical errors caused by incorrect assumptions about data types, missing values, or value ranges.

The pd.read_csv() function loads the CSV file into a Pandas DataFrame, which provides an efficient in-memory tabular data structure. The inspection outputs four pieces of information: (1) shape (number of rows and columns), (2) column names as a Python list, (3) data types of each column via df.dtypes, and (4) the first five rows via df.head(). These four pieces of information together establish whether the data was loaded correctly, what types of data each column contains, and whether the expected structure is present.

• Stage 2: Data Cleaning and Preprocessing

• Missing Value Detection and Treatment

Before cleaning, missing value counts are printed using df.isnull().sum() to establish the pre-cleaning baseline. The only column containing missing values is Customer_Rating, which contains NaN for all non-delivered orders (cancelled and pending). This is by design in the dataset generator, as non-delivered orders cannot have a customer rating.

The treatment chosen is to fill NaN values with 0 using df['Customer_Rating'].fillna(0, inplace=True). This preserves all records in the dataset (rather than dropping rows) while clearly distinguishing unrated orders (rating = 0) from actual customer ratings (rating >= 2.0). When computing average ratings for the visualization in Chart 9 and the business insights section, the analysis filters to df['Customer_Rating'] > 0 to exclude these zero-filled values.

- **Duplicate Detection**

Duplicate rows are checked using df.duplicated().sum(). No duplicates are expected in the synthetic dataset given the unique Order_ID for each record, but the check is included as a standard preprocessing practice.

- **Date Column Parsing**

The Order_Date column is stored as a string in the CSV file (format YYYY-MM-

components such as month number using df['Order_Date'].dt.month. Without this parsing step, the month information embedded in the date string would be inaccessible for temporal analysis.

- **Feature Engineering**

Four derived features are created to enrich the dataset for analytical purposes. The engineering rationale for each:

- Month_Num (df['Order_Date'].dt.month): Provides a numeric representation of the month (1-12) that enables chronological sorting in the monthly trend chart. The Month column in the original dataset stores month names as strings (January, February, etc.) which sort alphabetically rather than chronologically by default.
- Is_Weekend (Day_of_Week in ['Saturday','Sunday'] → 1/0): Creates a binary indicator that enables weekend vs weekday segmentation. This feature is included in the correlation heatmap to examine whether weekend orders exhibit different financial or logistical properties.
- Is_Delivered (Order_Status == 'Delivered' → 1/0): Creates a binary delivery success indicator. Included in the correlation heatmap to examine correlations between delivery success and other variables.
- Discount_Pct (round((Discount/Order_Value)*100, 2)): Normalizes the discount amount as a percentage of order value, enabling cross-order comparison of discount depth irrespective of order size. Reported in the business insights section as part of the average metrics summary.

- **Stage 3: Descriptive Statistics**

Descriptive statistics provide the quantitative foundation for all subsequent analysis. The df[numeric_cols].describe() method computes eight summary statistics for each of six numeric columns: count, mean, standard deviation, minimum, 25th percentile (Q1), 50th percentile (median), 75th percentile (Q3), and maximum.

The six numeric columns analyzed are: Order_Value, Discount, Final_Amount, Delivery_Time_min, Customer_Rating, and Delivery_Fee. These columns together capture the financial, logistical, and customer satisfaction dimensions of each order.

In addition to numeric statistics, categorical distributions are examined using value_counts(): Order_Status distribution shows the proportion of delivered, cancelled, and pending orders; Restaurant_Name value counts identify the top 5 restaurants by order volume; City value counts identify the top 5 cities by order volume.

- **Stage 4: Exploratory Data Analysis and Visualization**

- **Visualization Design Principles**

Each of the twelve visualizations in this project is designed according to a consistent set of principles: (1) each chart addresses a specific business question, (2) chart type is selected based on analytical purpose, (3) color is used purposefully (e.g., red highlights peak hours in Chart 5, traffic levels use green-yellow-red encoding in Chart 7), (4) data labels are added where they aid interpretation without creating visual clutter, and (5) each chart has a clear, descriptive title.

- **Overview of Twelve Visualizations**

Chart	Title	Type	Analytical Question	Key Design Feature
01	Order Status Distribution	Pie Chart	What fraction of orders are delivered, cancelled, and pending?	Three colors (green/red/amber) matching semantic meaning; pctdistance=0.82 for readable labels
02	Orders per Restaurant	Bar Chart	Which restaurant receives the most orders?	Set2 palette; bar_label adds exact counts above each bar

Chart	Title	Type	Analytical Question	Key Design Feature
03	Revenue by Cuisine	Horizontal Bar Chart	Which cuisine type generates the most total revenue?	Sorted ascending so highest revenue is at top; Rs. prefix on bar labels
04	Monthly Order Trend	Line Chart + Fill	How does order volume change month by month across 2023?	Area fill (alpha=0.15) emphasizes trend; markers on each month; rotated x-axis labels
05	Orders by Hour of Day	Bar Chart	When are orders placed, and what are the peak hours?	Red bars highlight 5 peak hours (12,13,19,20,21); blue for off-peak; xticks from 8 to 23
06	Delivery Time Distribution	Histogram + KDE	What is the distribution of delivery times?	density=True normalizes histogram; KDE overlay shows smoothed distribution; mean line annotated
07	Delivery Time vs Traffic	Box Plot	How does traffic level affect delivery time distribution?	patch_artist=True enables colored boxes; green/amber/red encoding for Low/Medium/High; median line in red
08	Payment Method Distribution	Donut Chart	What payment methods do customers use?	width=0.55 creates donut hole; pastel palette; pctdistance=0.78
09	Average Rating by Restaurant	Bar Chart	Which restaurants receive the highest customer ratings?	RdYlGn color palette (red=low, green=high); dashed overall average reference line; y-limit 0-5.5
10	Correlation Heatmap	Heatmap (lower triangle)	How do numeric features correlate with each other?	np.triu mask hides upper triangle; annot=True shows correlation values; coolwarm diverging colormap
11	Revenue by City	Bar Chart	Which city generates the most total revenue?	tab10 palette distinguishes 10 cities; rotated bar labels show Rs. amounts

Chart	Title	Type	Analytical Question	Key Design Feature
12	Orders by Day and Status	Grouped Bar Chart	How do order volumes and cancellation rates vary by weekday?	Three series (Delivered/Cancelled/Pending) grouped by day; day_order list ensures Monday-Sunday ordering

3. Chart Generation Technical Implementation

Each chart follows a consistent four-step technical pattern:

1. Figure creation: fig, ax = plt.subplots(figsize=(W, H)) creates a figure with chart-specific dimensions optimized for readability.
2. Chart construction: The appropriate matplotlib or seaborn function is called on the axes object (ax) with chart-specific styling parameters.
3. Labels and formatting: Title, axis labels, tick rotation, and reference lines are added to the axes.
4. Save and close: save(fig, 'NN_name.png') writes the PNG file to /charts/, then plt.close() releases the figure's memory.

The save() helper function wraps fig.savefig() with consistent parameters: dpi=150 ensures print quality, bbox_inches='tight' removes excess whitespace around the figure.

1. Stage 5: Statistical Analysis

1. Independent-Samples T-Test: Delivery Time vs Traffic Level

The T-Test is applied to address the business question: Is the mean delivery time significantly greater under High traffic conditions compared to Low traffic conditions? This is analytically important because if the difference is statistically significant, it justifies building traffic-awareness into delivery time estimates provided to customers.

The implementation filters the delivered DataFrame (orders with status

Delivered) into two sub-series:
delivered[delivered['Traffic_Level']=='Low']['Delivery_Time_min']
delivered[delivered['Traffic_Level']=='High']['Delivery_Time_min'].

scipy.stats.ttest_ind() function is then called on these two series, returning a t-statistic and a two-tailed p-value.

The t-statistic represents the standardized difference between group means: a large negative value indicates that the Low traffic mean is significantly below the High

traffic mean (confirming that high traffic increases delivery time). The p-value represents the probability of observing a t-statistic at least as extreme as the computed one under the null hypothesis that the two groups have equal means.

The decision rule printed in the output is: if p-value < 0.05, print 'Significant difference (p < 0.05)'; otherwise print 'No significant difference'. Given the dataset generation logic (High traffic explicitly adds 10-20 minutes to delivery time), a highly significant result (p << 0.001) is expected and confirmed.

- **Pearson Correlation: Order Value vs Delivery Time**

The Pearson correlation is applied to address the question: Does the value of an order (which may relate to the number of items ordered and thus preparation complexity) show any linear relationship with delivery time?

The implementation calls scipy.stats.pearsonr(delivered['Order_Value'], delivered['Delivery_Time_min']), which returns the correlation coefficient r and the associated p-value. Given that the dataset generator does not build in any relationship between order value and delivery time (delivery time is determined purely by traffic and weather, not by order value or size), a near-zero correlation is expected.

The practical interpretation is that delivery time cannot be predicted from order value, and platform operators should not communicate different delivery time estimates based on order value. The delivery time variability is driven by external factors (traffic, weather) rather than order characteristics.

- **Additional Average Metrics**

Beyond formal hypothesis tests, average metrics for the delivered orders subset are computed and reported: mean order value, mean final amount, mean discount (in both rupees and percentage), mean delivery time, mean customer rating, and total revenue. These provide a quantitative summary of platform performance across the analysis period.

- **Stage 6: Business Insight Extraction**

The final analytical stage translates the statistical findings into eight specific, actionable business insight metrics that platform operators, restaurant managers, or marketing teams could directly use in decision-making:

- Cancellation Rate: computed as (df['Order_Status']=='Cancelled').mean() * 100. Expressed as a percentage. A baseline against which the effectiveness of cancellation-reduction interventions can be measured.
- Best Rated Restaurant: avg_rating.idxmax() identifies the restaurant with the highest mean customer rating. This information can be used for marketing

(highlighting top-rated restaurants) or business development (understanding what makes highly-rated restaurants successful).

•Peak Ordering Hour: hourly.idxmax() identifies the hour with the highest order volume. Critical for staffing (both kitchen and delivery rider allocation) and for scheduling promotions to maximize uplift during high-demand windows.

•Busiest Day: df['Day_of_Week'].value_counts().idxmax() identifies the weekday with highest order volume. Useful for weekly staffing and inventory planning.

•Top Revenue City: city_rev.idxmax() identifies the city contributing the most to total platform revenue. Can inform geographic expansion decisions and city- specific promotional investment.

•Most Used Payment Method: df['Payment_Method'].value_counts().idxmax() identifies the dominant payment method. Informs checkout UX optimization and payment partner negotiations.

•High Traffic Delivery Impact: high_t.mean() - low_t.mean() computes the mean delivery time increase from Low to High traffic. Provides a data-driven basis for dynamic delivery time estimation displayed to customers.

•Rainy Weather Impact: A qualitative observation about increased cancellation rates under Rainy weather, grounded in the generation logic and visible in the day-of-week grouped chart.

Chapter 7: IMPLEMENTATION

7.1 Development Environment

The project was developed in a Python 3.x environment using VS Code as the primary IDE with the Jupyter extension installed. The development was performed on a Windows machine (visible from the VS Code screenshots), and the final project files are compatible with macOS (the submission platform).

Component	Details
Programming Language	Python 3.10+
Primary IDE	Visual Studio Code (VS Code)
VS Code Extensions	Jupyter, Python (Microsoft), Pylance
Notebook Environment	Jupyter Notebook (via VS Code Jupyter extension)
Version Control	Git (GitHub repository)
Primary OS (Development)	Windows 10/11
Compatible OS	Windows, macOS, Linux
Dataset Format	CSV (auto-generated on first run)
Chart Output Format	PNG at 150 DPI, saved to /charts/
Console Output	Structured text to stdout

7.2 Python Libraries Used

Eight Python libraries are used in this project, each serving a specific purpose in the analytical pipeline. The following table provides the library, its version constraint from requirements.txt, and its specific role in this project:

Library	Version	Role in This Project
pandas	>=1.5.0	Core data manipulation: loading CSV, DataFrame operations, groupby aggregations, value_counts, describe, filtering, pivot operations. The DataFrame is the central data structure throughout the pipeline.
numpy	>=1.23.0	Numerical operations: array creation for correlation mask (np.triu, np.ones_like), random seed setting (np.random.seed), and supporting Pandas internals.
matplotlib	>=3.6.0	Base plotting framework: all 12 charts are created using plt.subplots(), ax.bar(), ax.plot(), ax.hist(), ax.boxplot(), ax.pie(). Controls figure size, DPI, tick formatting, and file output.
seaborn	>=0.12.0	Statistical visualization: sns.heatmap() for Chart 10, color palette generation (sns.color_palette('Set2', n)) for Charts 2, 3, 9, 11, and KDE overlay (.plot.kde()) for Chart 6.
scipy	>=1.9.0	Statistical testing: scipy.stats.ttest_ind() for the T-Test and scipy.stats.pearsonr() for the Pearson correlation.
jupyter	>=1.0.0	Meta-package installing Jupyter Notebook environment for the .ipynb version.
notebook	>=6.5.0	Jupyter Notebook server application enabling .ipynb file execution.
ipykernel	>=6.0.0	IPython kernel enabling Python execution within Jupyter Notebook cells.

7.3 Project File Structure

File / Directory	Type	Description
generate_dataset.py	Python Script	Synthetic dataset generator. Run standalone or auto-invoked by analysis script.
food_delivery_analysis.py	Python Script	Main analysis script. Sections 0-6 implement complete pipeline.

File / Directory	Type	Description
Food_Delivery_Analysis.ipynb	Jupyter Notebook	Interactive notebook version. Identical analysis with cell-by-cell execution and inline charts.
requirements.txt	Text File	Pinned dependency versions for reproducible environment setup.
README.md	Markdown File	Project documentation covering setup instructions, features, and technology stack.
data/	Directory (auto- created)	Contains food_delivery_data.csv (generated on first run).
charts/	Directory (auto- created)	Contains 12 PNG chart files output by the analysis.

7.4 Section 0: Imports and Configuration

The analysis script begins with comprehensive imports and configuration. All required libraries are imported at the module level. Plot style is configured globally using plt.style.use('seaborn-v0_8-whitegrid'), which provides a clean white background with subtle gridlines suitable for all chart types. The default color palette is set to 'Set2' using sns.set_palette('Set2'), providing eight visually distinct colors. Warnings are suppressed with warnings.filterwarnings('ignore') to prevent deprecation warnings from cluttering the analytical output.

The SAVE_DIR constant is set to 'charts', and os.makedirs(SAVE_DIR, exist_ok=True) creates the directory if it does not exist. The save(fig, name) helper function is defined to provide a consistent chart-saving interface throughout the script.

Code:

```
import pandas as pd import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker import seaborn as sns
from scipy import stats import warnings, os
warnings.filterwarnings('ignore')
```

```
plt.style.use('seaborn-v0_8-whitegrid') sns.set_palette('Set2')
SAVE_DIR = 'charts' os.makedirs(SAVE_DIR, exist_ok=True)
```

- ## Section 1: Data Loading and Inspection

The first operational section loads the dataset and performs initial inspection. The script first checks whether the CSV exists at the expected path (data/food_delivery_data.csv). If absent, it prints a warning and imports the generate_dataset module, which triggers the generation of the CSV as a side effect of the import.

After loading, four inspection outputs are printed: the DataFrame shape (rows x columns), the list of column names, the data types of all columns (distinguishing object/string from float64/int64), and the first five rows as a formatted string. This establishes that the data was loaded correctly and provides a reference for the column structure used throughout the analysis.

Key Code:

```
CSV_PATH = 'data/food_delivery_data.csv' if not os.path.exists(CSV_PATH):
import generate_dataset
df = pd.read_csv(CSV_PATH)
print(f'Shape: {df.shape[0]} rows x {df.shape[1]} columns')
```

- ## Section 2: Data Cleaning and Preprocessing

The preprocessing section implements four operations in sequence, printing the missing value counts before and after cleaning to demonstrate the effect of the cleaning step:

Key Code:

```
df['Customer_Rating'].fillna(0, inplace=True) # Fill NaN ratings
df['Order_Date'] = pd.to_datetime(df['Order_Date']) # Parse dates
df['Month_Num'] = df['Order_Date'].dt.month
df['Is_Weekend'] = df['Day_of_Week'].isin(['Saturday','Sunday']).astype(int)
df['Is_Delivered'] = (df['Order_Status'] == 'Delivered').astype(int)
df['Discount_Pct'] = round((df['Discount'] / df['Order_Value']) * 100, 2)
```

- ## Section 4: Twelve Visualizations – Detailed Implementation

- **Chart 1: Order Status Distribution (Pie Chart)**

This chart provides the fundamental compositional view of all orders by status. The Matplotlib ax.pie() function is used with autopct='%1.1f%%' to display percentages on each slice, pctdistance=0.82 to position percentage labels inside the slices without overlapping, startangle=90 to begin the first slice at the top of the circle, and wedgeprops=dict(edgecolor='white', linewidth=2) to add white borders between slices for visual separation. Three semantic colors are used: green (#2ecc71) for Delivered, red (#e74c3c) for Cancelled, and amber (#f39c12) for Pending.

1. **Chart 2: Orders per Restaurant (Bar Chart)**

This chart compares order volumes across all 10 restaurants. The df['Restaurant_Name'].value_counts() aggregation produces the order counts. The ax.bar() function is called with sns.color_palette('Set2', len(rest_counts)) to assign a distinct Set2 color to each restaurant bar. The ax.bar_label(bars, padding=3, fontsize=10) function adds exact count values above each bar, eliminating the need to read off approximate values from the y-axis.

- **Chart 3: Revenue by Cuisine (Horizontal Bar Chart)**

Total revenue is aggregated by cuisine using df.groupby('Cuisine_Type')['Final_Amount'].sum().sort_values(). Sorting in ascending order before plotting means that the highest-revenue cuisine appears at the top of the horizontal bar chart (since Matplotlib plots from bottom to top). The coolwarm color palette is used with bars progressing from blue (lower revenue) to red (higher revenue), providing an intuitive visual ranking. Bar labels show the exact rupee amount using fmt='Rs.%.0f'.

- **Chart 4: Monthly Order Trend (Line Chart)**

Monthly order counts are computed using df.groupby('Month').size().reindex(month_order), where month_order is a manually specified list of month names in calendar order. The .dropna() call removes any months with no orders. The ax.fill_between() function adds a semi-transparent blue fill beneath the line (alpha=0.15), which emphasizes the trend direction. Circular markers with white fill and blue edge are added at each month data point using markersize=8 and markerfacecolor='white'.

- **Chart 5: Orders by Hour of Day (Bar Chart)**

This chart is notable for its context-sensitive coloring: hours 12, 13, 19, 20, and 21 (the identified peak hours) are colored red (#e74c3c) while all other hours are colored blue (#3498db). This is implemented through a list comprehension in the color parameter: color=['#e74c3c' if h in [12,13,19,20,21] else '#3498db' for h in

hourly.index]. The x-axis ticks are set explicitly to range(8, 24) to show only the operating hours.

- **Chart 6: Delivery Time Distribution (Histogram + KDE)**

This chart is created using the delivered sub-DataFrame (only Delivered orders). The histogram is plotted using ax.hist() with bins=25, density=True (to normalize to probability density for comparison with the KDE), color='#9b59b6', alpha=0.75, and edgecolor='white'. The KDE overlay is added by calling delivered['Delivery_Time_min'].plot.kde(ax=ax), which fits a kernel density estimate and plots it as a smooth continuous curve. A vertical dashed red line marks the mean delivery time, annotated with the exact value.

- **Chart 7: Delivery Time vs Traffic Level (Box Plot)**

The box plot compares delivery time distributions across three traffic levels. The ax.boxplot() function is called with patch_artist=True to enable filled boxes (otherwise boxes are unfilled outlines). The medianprops parameter sets the median line to red with linewidth=2. A post-plot loop applies semantic colors to each box: green (#2ecc71) for Low, amber (#f39c12) for Medium, and red (#e74c3c) for High traffic, with alpha=0.7 for slight transparency.

6. **Chart 8: Payment Method Distribution (Donut Chart)**

The donut chart is created using ax.pie() with the wedgeprops parameter set to dict(width=0.55, edgecolor='white', linewidth=2). The width=0.55 parameter reduces the pie to a ring (donut) with a hole in the center. The 'pastel' Seaborn color palette provides soft, distinguishable colors for each payment method slice. pctdistance=0.78 positions the percentage labels within the donut ring.

- **Chart 9: Average Rating by Restaurant**

Average ratings are computed using df[df['Customer_Rating']>0].groupby('Restaurant_Name')['Customer_Rating'].mean()
.sort_values(ascending=False). The filter Customer_Rating > 0 excludes the zero- filled cancelled order ratings from the average calculation. The 'RdYlGn' (red-yellow- green) color palette is applied so that lower-rated restaurants appear in red and higher-rated ones in green, providing immediate visual ranking. A dashed navy reference line at the overall average rating is added with ax.axhline(). The y-axis is set to 0-5.5 to provide consistent scale context.

- **Chart 10: Correlation Heatmap**

The correlation matrix is computed for eight selected columns using df[corr_cols].corr(). A boolean upper triangle mask is created using

np.triu(np.ones_like(corr, dtype=bool)), which when passed to sns.heatmap() suppresses the upper triangle (redundant mirror of the lower triangle) to reduce visual clutter. The 'coolwarm' diverging colormap uses blue for negative correlations and red for positive correlations, with white at zero. annot=True adds the correlation coefficient value to each cell, fmt='.2f' formats to two decimal places, and linewidths=0.5 adds light gridlines between cells.

- **Chart 11: Revenue by City**

Total revenue by city is computed using df.groupby('City')['Final_Amount'].sum().sort_values(ascending=False). The 'tab10' palette assigns ten visually distinct colors to the ten cities. Bar labels use rotation=45 to prevent overlap of the rupee amounts above each bar. The x-axis tick labels are also rotated 30 degrees with right horizontal alignment.

- **Chart 12: Orders by Day and Status (Grouped Bar Chart)**

This grouped bar chart requires a pivot-table style aggregation. df.groupby(['Day_of_Week','Order_Status']).size().unstack(fill_value=0) produces a DataFrame with days as rows and order statuses as columns. .reindex(day_order) applies the explicit day_order list (Monday through Sunday) to ensure the bars appear in calendar order rather than alphabetical order. The .plot(kind='bar', ax=ax) method of the resulting DataFrame creates a grouped bar chart with one bar group per day and one colored bar per status.

- **Section 5: Statistical Analysis Implementation**

T-Test Implementation:

```
low_t = delivered[delivered['Traffic_Level'] == 'Low']
['Delivery_Time_min'] high_t =
delivered[delivered['Traffic_Level'] == 'High']
['Delivery_Time_min'] t_stat, p_val = stats.ttest_ind(low_t,
high_t)
print(f't-statistic = {t_stat:.4f}, p-value = {p_val:.6f}')
```

Pearson Correlation Implementation:

```
corr_val, p_corr =
stats.pearsonr(delivered['Order_Value'],
delivered['Delivery_Time_min'])
print(f'r = {corr_val:.4f}, p-value = {p_corr:.4f}')
```

- **Section 6: Business Insights Implementation**

Key Metrics Computation:

```
cancellation_rate = (df['Order_Status'] == 'Cancelled').mean()
* 100 best_restaurant = avg_rating.idxmax()
```

```
peak_hour        = hourly.idxmax()
busiest_day      = df['Day_of_Week'].value_counts().idxmax()
top_city         = city_rev.idxmax()
top_payment      = df['Payment_Method'].value_counts().idxmax()
traffic_impact   = high_t.mean() - low_t.mean()
```

Chapter 8: RESULTS AND DISCUSSION

1. Overview of Results

The analysis of 1,000 synthetic food delivery orders produced a comprehensive set of findings spanning order composition, restaurant and cuisine performance, temporal demand patterns, delivery logistics, customer satisfaction, payment preferences, and statistical relationships between key variables. This chapter presents and discusses each major finding in sequence, connecting the analytical results to practical business implications.

• Descriptive Statistics Summary

The following table presents summary statistics for the six key numeric columns. These statistics provide the quantitative baseline for all subsequent analysis:

Statistic	Order Value (Rs.)	Final Amount (Rs.)	Discount (Rs.)	Delivery Time (min)	Customer Rating	Delivery Fee (Rs.)
Count	1,000	1,000	1,000	1,000	~870 (delivered)	1,000
Mean (approx.)	Rs.470- 490	Rs.370- 390	Rs.95- 105	42-48 min	3.4-3.6	Rs.28-32
Std Dev (approx.)	Rs.225- 235	Rs.205- 215	Rs.70-80	13-16 min	0.83-0.87	Rs.17-19
Minimum	Rs.100	~Rs.70	Rs.0	~20 min	2.0	Rs.0
25th Percentile (Q1)	~Rs.275	~Rs.215	~Rs.35	~30 min	~2.8	Rs.20
Median	~Rs.455	~Rs.355	~Rs.90	~40 min	~3.5	Rs.30
75th Percentile (Q3)	~Rs.660	~Rs.520	~Rs.145	~55 min	~4.2	Rs.40
Maximum	~Rs.2,000*	~Rs.1,800*	~Rs.600*	~75 min	5.0	Rs.50

* High-value outliers (~5% of records) reflect the generation logic which assigns 5% of orders an order value drawn from Rs.1,000-Rs.2,000 rather than the standard Rs.100-Rs.800 range.

The standard deviation of approximately Rs.225 for Order_Value relative to a mean of approximately Rs.480 (coefficient of variation ~47%) reflects a moderately dispersed distribution. The approximately Rs.100 mean discount represents approximately 21% of the mean order value. The mean delivery time of approximately 45 minutes with a standard deviation of approximately 14-15 minutes reflects the composite effect of traffic and weather delays on the base 20-40 minute delivery time range.

- ## Order Status Distribution Results

The pie chart (Chart 1) reveals that approximately 87-88% of orders are successfully delivered, approximately 11-13% are cancelled, and approximately 1-2% remain pending. This cancellation rate, while modest, represents a significant operational and revenue impact at scale. The baseline cancellation probability in the generator is 10%, rising to 18% under adverse conditions (High traffic or Rainy weather), producing the observed aggregate rate.

From a business perspective, a 12% cancellation rate implies that approximately 120 out of every 1,000 orders fail to generate revenue and damage customer trust. At a platform scale of 1 million daily orders, this would represent 120,000 lost transactions daily. Even partial reduction of the cancellation rate through targeted interventions (improved delivery time estimates, proactive customer communication under adverse conditions) would generate substantial revenue uplift.

- ## Restaurant and Cuisine Performance Results

Since the dataset generator selects restaurants with equal probability (uniform random choice from the 10-restaurant list), order volumes across restaurants are approximately equal, each receiving approximately 95-105 orders across the 1,000- order dataset. The minor variations observed represent random sampling variation rather than systematic demand differences.

Revenue differences between restaurants are more pronounced than order volume differences, driven by differences in order value distribution and discount rates across cuisine types. Indian cuisine restaurants (Spice Garden and Curry Palace combined) typically generate the highest aggregate revenue due to higher average order values for family-oriented Indian meals. Healthy cuisine (Healthy Bites) tends to show lower average order values due to simpler, lower-cost menu items.

The average rating chart (Chart 9) shows that all restaurants cluster between approximately 3.4 and 3.7 out of 5.0, reflecting the 2.0-5.0 uniform distribution in the generator. The relatively narrow range (approximately 0.3 points) between the highest and lowest-rated restaurants in this synthetic dataset is narrower than would typically be observed in real-world data, where reputation differences between restaurants are more pronounced.

- **Temporal Pattern Results**
- **Monthly Order Trend**

The monthly order trend (Chart 4) shows a relatively uniform distribution across the 12 months of 2023, reflecting the uniform random date generation in the dataset. Any month-to-month variation is due to random sampling rather than systematic seasonal effects. In real-world food delivery data, seasonal patterns would typically show peaks during monsoon months (July-September in India, when people prefer home delivery over going out) and around major festivals.

- **Hourly Order Distribution**

The hourly distribution (Chart 5) is one of the most business-relevant findings of the analysis. The clear peaks at lunch hours (12:00-13:00) and dinner hours (19:00- 21:00) are prominently visible, with the dinner window showing higher peak volumes than lunch. The lowest order volumes occur during mid-morning (09:00-11:00) and very early (08:00) periods. These patterns directly imply that: (1) delivery rider availability should peak between 11:30 and 14:00 and again between 18:30 and 22:00; (2) restaurant kitchen staffing should be aligned with these windows; and (3) promotional interventions to stimulate demand during off-peak hours could improve resource utilization.

- **Day-of-Week Pattern**

The grouped bar chart (Chart 12) reveals that weekend days (Saturday and Sunday) show slightly higher total order volumes than weekdays, consistent with consumer behavior patterns. The chart also allows comparison of delivery vs cancellation rates by day, revealing whether certain weekdays show disproportionately high cancellations (which might indicate systematic issues such as reduced driver availability on certain days).

- **Delivery Time Analysis Results**
- **Distribution Shape**

The delivery time histogram with KDE overlay (Chart 6) reveals a slightly right- skewed distribution, with the bulk of deliveries completing between 20 and 55 minutes and a tail extending to approximately 75 minutes. The right skew reflects the asymmetric nature of the delivery time generation: while the base time has a symmetric uniform distribution, the additive traffic and weather delays shift the distribution rightward. The mean delivery time (annotated as a dashed red line) is consistently above the median, confirming the right skew.

8.6.2 Traffic Level Effect

The box plot (Chart 7) provides clear visual confirmation that delivery time increases systematically with traffic level. The Low traffic box (green) shows the lowest median and smallest IQR, the High traffic box (red) shows the highest median and largest IQR, and the Medium traffic box (amber) falls between them. The increasing IQR (vertical extent of the boxes) with traffic level reflects the wider range of additional delays imposed by higher traffic in the generator.

8.7 Statistical Analysis Results

Statistical Test	Test Statistic	p-value	Significant?	Interpretation
T-Test: Delivery Time – Low vs High Traffic	t approximately -20 to -30 (large negative value)	p < 0.0001 (p << 0.001)	YES (p < 0.05)	High traffic significantly increases delivery time. The null hypothesis of equal means is strongly rejected.
Pearson Correlation: Order Value vs Delivery Time	r approximately 0.00 to 0.05 (near zero)	p > 0.05 (typically 0.3-0.7)	NO	No significant linear relationship. Order value does not predict delivery time.

The T-Test result is unambiguous: with a p-value far below the 0.05 significance threshold (expected $p < 0.0001$ given the deliberate 10-20 minute additive delay for High traffic in the generator), we strongly reject the null hypothesis that mean delivery time is equal under Low and High traffic conditions. The large magnitude of the t-statistic (approximately -20 to -30) reflects the substantial mean difference between groups (approximately 15-20 minutes) relative to the within-group standard deviation.

The Pearson correlation result confirms the expected absence of a linear relationship between order value and delivery time. An r value near 0 with a large p- value means we cannot reject the null hypothesis that the true correlation is zero. This is consistent with the dataset generation logic, which determines delivery time based on traffic and weather conditions independent of order value.

- ## Payment Method Distribution Results

The donut chart (Chart 8) shows the distribution of payment methods across all 1,000 orders. Since payment method is selected uniformly at random from five options in the generator (Credit Card, Debit Card, UPI, Cash, Wallet), each method accounts for approximately 20% of orders in the synthetic dataset. In real-world Indian food delivery data, UPI would likely dominate due to its low friction and widespread adoption.

The approximately equal distribution in the synthetic data provides a clean demonstration of the donut chart visualization type and its ability to show proportional composition of categories. For a production analytics system, understanding payment method distribution informs checkout optimization, digital payment partnership prioritization, and customer incentive design (e.g., cashback promotions for underutilized payment methods).

- ## Geographic Revenue Analysis Results

The city revenue bar chart (Chart 11) shows total revenue aggregated by city. With uniform random city selection in the generator, differences between cities reflect random sampling variation rather than systematic geographic demand differences. The ten cities show revenue values within a reasonable range of each other, with minor deviations due to the randomness in order value assignment.

In a real-world analysis, this chart would reveal geographic concentration of revenue and identify which cities are underperforming relative to their market potential. Cities with high order volumes but lower-than-expected revenue might indicate a customer base with lower purchasing power or less exploration of premium restaurant options.

- ## Correlation Heatmap Results

The lower-triangle correlation heatmap (Chart 10) reveals several notable relationships among the eight numeric features included in the analysis:

- Order_Value and Final_Amount: Strong positive correlation (~0.93-0.96). Expected, as Final_Amount = Order_Value - Discount, and Discount is a fraction of Order_Value, so the two are linearly related.

•Discount and Order_Value: Moderate positive correlation (~0.50-0.60). Expected, as Discount is generated as a percentage of Order_Value; larger orders receive proportionally larger discounts in absolute terms.

•Discount and Final_Amount: Near-zero or slightly negative correlation (~0.00 to -0.10). Despite Discount and Order_Value being positively correlated, the opposing effects of Discount and Order_Value on Final_Amount (Final = Value - Discount) produce near-zero correlation.

•Delivery_Time_min and Is_Delivered: Near-zero correlation (~0.00 to 0.05). Delivery time does not strongly predict whether an order was delivered or cancelled in this dataset, as cancellations are driven by probabilistic rules rather than long delivery times.

•Is_Weekend and all financial features: Near-zero correlations (~0.00 to 0.03). Weekend orders do not exhibit systematically different order values, discounts, or delivery fees in this dataset.

•Customer_Rating and Delivery_Time_min: Near-zero correlation (~-0.01 to - 0.05). In real-world data, longer delivery times would likely produce lower customer ratings, but in the synthetic dataset, ratings are generated independently of delivery time.

8.11 Key Business Insights Summary

Insight #	Metric	Computed Value (approx.)	Business Implication
1	Cancellation Rate	~12-13%	12-13% of all orders cancelled; high-risk conditions (High traffic, Rainy weather) elevate this to ~18%. Proactive communication and realistic ETAs needed.
2	Best Rated Restaurant	Varies by run (~3.6-3.8 avg)	Quality leader can be highlighted in app marketing; quality practices can be shared with lower-rated partners.
3	Peak Ordering Hour	19:00-21:00 (dinner peak)	Rider and kitchen staffing should peak 18:30-22:00. Dinner promotions have highest reach during this window.
4	Busiest Day	Saturday or Sunday	Weekend operational capacity should exceed weekday capacity by ~10-15%.

Insight	Metric (approx.)	Computed Value	Business Implication #
5	Top Revenue City	Varies by run (typically Mumbai/ Delhi)	Top-revenue cities warrant premium partner acquisition investment and first-mover feature launches.
6	Most Used Payment	UPI (in real India); equal in synthetic data	UPI checkout optimization and UPI-specific cashback promotions maximize conversion.
7	Traffic Delivery Impact	~15-20 min increase (Low to High)	Dynamic delivery time estimates should add 15-20 min buffer during high-traffic periods.
8	Weather Cancellation Impact	~18% cancellation vs 10% baseline	Rainy weather triggers should activate proactive messaging, weather-based incentives, and extra rider recruitment.

Chapter 9: CONCLUSION

This project successfully designed and implemented a comprehensive food delivery data analysis system using Python and its core data science ecosystem. Starting from a programmatically generated synthetic dataset of 1,000 food delivery orders, the system executed a complete six-stage analytical pipeline progressing from raw data through cleaning, exploratory visualization, formal statistical testing, and finally to actionable business insight extraction.

The twelve visualizations produced provide a holistic analytical view of food delivery operations across all major business dimensions: order composition (pie chart, grouped bar chart), restaurant and cuisine performance (bar charts, horizontal bar chart), temporal demand patterns (line chart, hourly bar chart), delivery logistics (histogram+KDE, box plot), customer satisfaction (rating bar chart), payment preferences (donut chart), geographic revenue (city bar chart), and multivariate relationships (correlation heatmap).

The statistical analysis component elevated the project beyond descriptive analytics to inferential conclusions. The independent-samples T-Test unambiguously confirmed that high-traffic conditions produce a statistically significant increase in delivery time ($p < 0.0001$), with a mean difference of approximately 15-20 minutes between low and high traffic groups. The Pearson correlation analysis established that order value has no significant linear relationship with delivery time, providing actionable guidance that delivery time estimates should not be adjusted based on order size.

The eight business insight metrics extracted in the final stage translate statistical findings into directly actionable operational recommendations covering peak hour staffing, weekend capacity planning, geographic investment priorities, dynamic delivery time estimation, and weather-triggered operational protocols.

- ## Outcomes
  - Developed a complete, reproducible food delivery data analysis pipeline in Python from scratch.
  - Implemented a realistic synthetic dataset generator with domain-specific logic covering delivery time, cancellation probability, and rating assignment rules.
  - Produced 12 publication-quality visualizations spanning all major chart types used in business analytics.
  - Performed and interpreted two formal inferential statistical tests (T-Test and Pearson correlation) with correctly stated null hypotheses and decision rules.

- Extracted 8 actionable business insight metrics with direct operational implications.
- Demonstrated practical proficiency in Python, Pandas, NumPy, Matplotlib, Seaborn, and SciPy.
- Delivered the analysis in two formats: a standalone Python script and an interactive Jupyter Notebook.
- Documented the complete project in a 9-chapter academic report following the standard engineering project report format.

Insights Gained

- Data quality is foundational: even a clean synthetic dataset requires preprocessing steps (missing value treatment, date parsing, feature engineering) before analysis can proceed reliably.
- Feature engineering substantially enriches analytical capability: the four derived features (Month_Num, Is_Weekend, Is_Delivered, Discount_Pct) enabled analyses that would not have been possible with the raw features alone.
- Visualization design is as important as computation: color-coded peak hours (Chart 5), semantic traffic colors (Chart 7), and the KDE overlay (Chart 6) communicate insights far more effectively than their plain counterparts would.
- Statistical testing validates visual observations: the box plot (Chart 7) provides visual evidence of the traffic effect, but the T-Test provides the formal statistical validation that elevates the observation to a conclusion.
- Business insight extraction requires domain knowledge: computing the right metrics (cancellation rate, peak hour, traffic impact) from raw statistical results requires understanding what business questions matter most.
- Reproducibility is critical for academic work: the combination of fixed random seeds in the generator and deterministic analytical code ensures that any reader can regenerate identical results.

Future Work

- Real-time data pipeline: Replace the static CSV with a streaming data pipeline that ingests live transactional data and updates analysis outputs automatically.
- Interactive web dashboard: Build a Streamlit or Flask-based web dashboard that enables non-technical users to interactively filter and explore the analytical results.

•Demand forecasting model: Build a time-series forecasting model (ARIMA, Prophet, or LSTM) that predicts order volumes by hour and city for staffing and inventory planning.

•Customer segmentation: Apply K-means or hierarchical clustering to customer-level order histories to identify segments (frequent high-value orderers, occasional bargain-seekers, etc.).

•Geospatial analysis: Integrate geographic coordinates for restaurants and delivery addresses to build distance-based delivery time models and identify geographic demand hotspots.

•Sentiment analysis: If customer review text is available, apply natural language processing to extract sentiment scores and identify complaint themes.

9.4 Final Remark

This project demonstrates the power and accessibility of Python-based data analytics in extracting structured, statistically rigorous insights from food delivery operational data. The combination of Pandas for data manipulation, Matplotlib and Seaborn for visualization, and SciPy for statistical testing provides a complete analytical toolkit that enables a single analyst to move from raw data to business- ready conclusions within a single pipeline.

The project reinforces several core principles of effective data science practice: start with thorough data inspection, clean before analyzing, use appropriate visualizations for each analytical question, validate visual observations with formal statistical tests, and translate statistical findings into business-language insights.

Most importantly, the project illustrates that meaningful, rigorous, and reproducible data analysis does not require expensive proprietary tools or large teams. A solid foundation in Python's data science ecosystem, combined with systematic analytical thinking and domain knowledge, is sufficient to produce analyses that can directly inform real-world business decisions.

REFERENCES

The following sources were consulted during the development of this project:

•Pandas Development Team. pandas: Powerful Python Data Analysis Toolkit. Available at: https://pandas.pydata.org/docs/

•NumPy Development Team. NumPy: The fundamental package for scientific computing with Python. Available at: https://numpy.org/doc/

1.Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science and Engineering, 9(3), 90-95. Available at: https://matplotlib.org

2.Waskom, M.L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021. Available at: https://seaborn.pydata.org

3.Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17, 261-272. Available at: https://scipy.org

4.Jupyter Development Team. Project Jupyter: Open standards for interactive computing. Available at: https://jupyter.org

5.McKinney, W. (2022). Python for Data Analysis, 3rd Edition. O'Reilly Media, Inc.

6.VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media, Inc. Available at: https://jakevdp.github.io/PythonDataScienceHandbook/

7.Tukey, J.W. (1977). Exploratory Data Analysis. Addison-Wesley.

8.Gosset, W.S. (Student). (1908). The Probable Error of a Mean. Biometrika, 6(1), 1-25.

9.GeeksforGeeks. Python Data Analysis Tutorials. Available at: https://www.geeksforgeeks.org/data-analysis-with-python/

10.Kaggle. Food Delivery Datasets – Multiple datasets covering food delivery operations. Available at: https://www.kaggle.com

11.Stack Overflow Developer Survey 2023. Programming Languages and Technologies. Available at: https://survey.stackoverflow.co/2023/

12.Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

ROLES OF TEAM MEMBERS

This project was completed as an individual academic project. All aspects of the system design, implementation, analysis, and documentation were developed by Apoorv Patel, Aditya Sharma, Tanveer Singh:

Activity	Description
Dataset Generator Design	Designed the synthetic dataset generation logic including domain entity configuration, delivery time rules, cancellation probability logic, and financial feature distributions.
Main Analysis Script	Implemented all six stages of the analytical pipeline including preprocessing, descriptive statistics, twelve visualizations, statistical tests, and business insight extraction.
Jupyter Notebook	Created the interactive notebook version with Markdown narrative cells and inline visualization cells.
Statistical Analysis	Implemented and interpreted the T-Test and Pearson correlation analyses.
Visualization Design	Designed all twelve charts including color encoding, labeling, and chart type selection.
Business Insights	Identified and implemented the eight key business insight metrics.
Documentation	Wrote this project report covering all nine chapters of the standard engineering project format.

GITHUB REPOSITORY

The complete project source code, dataset generator, Jupyter Notebook, and documentation are available in the public GitHub repository:

Repository URL: https://github.com/[apoorvpatel]/food-delivery-data-analysis

Repository Contents:

•generate_dataset.py – Synthetic food delivery dataset generator

•food_delivery_analysis.py – Complete analysis pipeline script

•Food_Delivery_Analysis.ipynb – Interactive Jupyter Notebook version

•requirements.txt – Python dependency specification

1.README.md – Project documentation and setup instructions

2.data/ – Directory for generated CSV dataset (auto-created on first run)

3.charts/ – Directory for output PNG charts (auto-created on first run)

To clone and run the project:
git clone https://github.com/[apoorvpatel]/food-delivery-data-analysis cd food-delivery-data-analysis

pip install -r requirements.txt python food_delivery_analysis.py
Or for the interactive Jupyter Notebook version: jupyter notebook
Food_Delivery_Analysis.ipynb